# Computer Graphics Project 1: Path Tracer (Labs 1–4)

## Introduction

The goal of this project was to implement, over the course of four labs, a path tracer in C++ using a template provided by the professor. The four sections were split as such: in Lab 1, we implemented diffuse and mirror surfaces. In Lab 2, we developed a more advanced system with indirect lighting and antialiasing. In Lab 3, we built the support for complex meshes, with the implementation of the Moller-Trumbore algorithm for mesh rendering. Finally, in Lab 4, we added BVH support for better optimisation.

The project was run on my own machine, a Dell XPS15 with Intel Core i9-13900H CPU and NVIDIA RTX4060 Laptop GPU. This is important since it explains which some of our runtimes are so low.

## 1 Lab 1: Diffuse and mirror surfaces, direct lighting, hard shadows

In Lab 1, we built the first working version of the ray tracer. The idea was the following: for every pixel of the image, we shoot a ray from the camera, find the closest object it hits, and decide what color the pixel should be.

We first wrote the ray–sphere intersection. We subsitute the ray $\mathbf{P}(t) = \mathbf{O} + t\mathbf{u}$ into the sphere equation $\|\mathbf{P} - \mathbf{C}\|^2 = R^2$, which gives a quadratic in $t$, and we keep the smallest positive root. The scene's nearest-hit function then loops over all objects and keeps the closest one.

Once we know what was hit, we compute the direct lighting from the point light at $L$ with intensity $I$:

$$L_{\text{direct}}(\mathbf{P}) = \frac{I}{4\pi \|L - P\|^2} \cdot \frac{\rho}{\pi} \cdot < \mathbf{N}, \frac{L - P}{||L - P||} >$$

To determine shadows, we shoot another ray from the point we hit. If anything blocks it, then the point is in shadow.

For mirror surfaces, we directly perform recurisve calls to trace a reflected ray in the direction:

$$\mathbf{R} = \mathbf{u} - 2(\mathbf{u} \cdot \mathbf{N})\,\mathbf{N}.$$

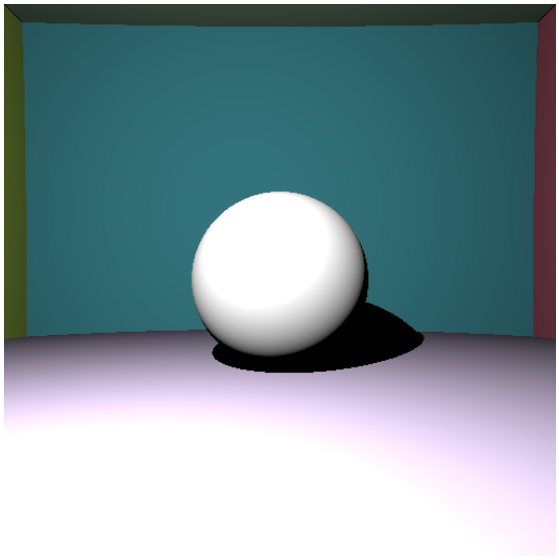


Figure 1: Lab 1 render. $512 \times 512$, 1 sample/pixel, $\gamma = 2.2$, $I = 3 \times 10^7$. Render time: **0.03 s**.

## 2 Lab 2: Indirect lighting and antialiasing

In Lab 2, we added two important features: indirect lighting (light bouncing between surfaces) and antialiasing (smoothing out jagged pixel edges).

For indirect lighting, we estimate the rendering equation with Monte Carlo. We sample one random direction in the hemisphere using a cosine-weighted PDF. The bounce ray's radiance $L_i$ is obtained by recursively calling `getColor`, and we ensure termination by capping the number of light bounces to 5.

For antialiasing, instead of shooting through the centre of each pixel, we average several rays per pixel. We do this using the Box–Muller transform. With 32 samples per pixel, the Monte Carlo noise from the indirect lighting is also clearly reduced, since variance goes down as $1/N$.

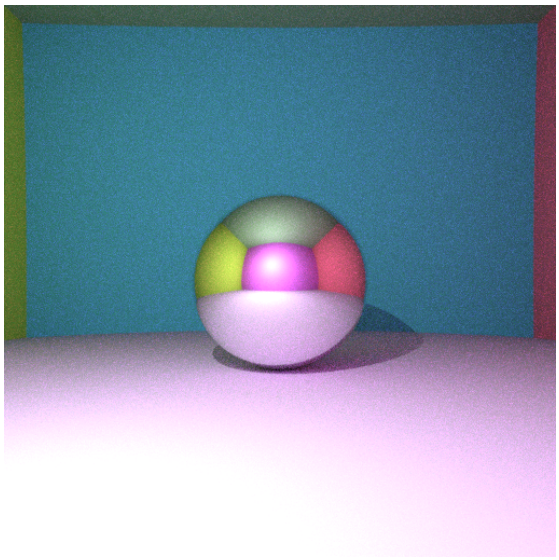


Figure 2: Lab 2 render. $512 \times 512$, 32 spp, 5 bounces, $\gamma = 2.0$, $I = 2 \times 10^7$. The walls clearly tint each other through the indirect bounce (colour bleed), and pixel edges are smoothed. Render time: **1.36 s**.

# 3 Lab 3: Triangle meshes with Moller–Trumbore and AABB acceleration

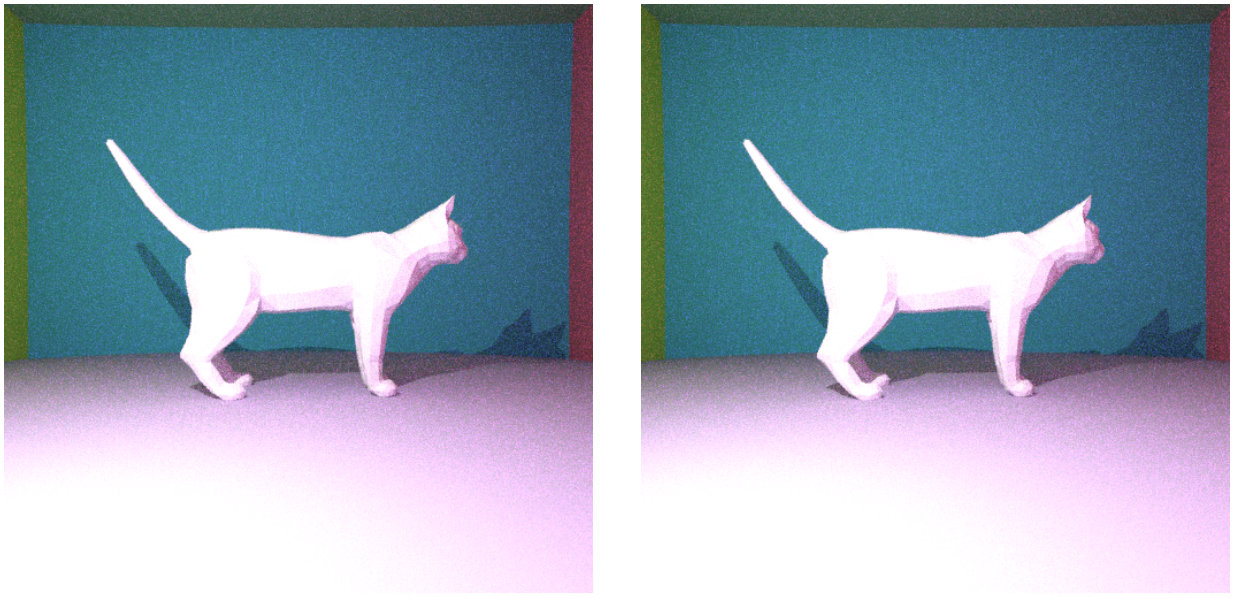
In Lab 3, we added support for arbitrary triangle meshes. We tested it on the provided `cat.obj` model.

The core piece is the Moller–Trumbore algorithm for ray–triangle intersection. We set the ray equation equal to the barycentric form of the triangle, $\mathbf{O}+t\mathbf{u} = \alpha\mathbf{A}+\beta\mathbf{B}+\gamma\mathbf{C}$ with $\alpha+\beta+\gamma = 1$, and solve for the unknowns. Writing $\mathbf{e}_1 = \mathbf{B} - \mathbf{A}$, $\mathbf{e}_2 = \mathbf{C} - \mathbf{A}$, $\mathbf{N}_T = \mathbf{e}_1 \times \mathbf{e}_2$, and $\mathbf{A}_O = \mathbf{A} - \mathbf{O}$ (as done in the code I wrote), we get

$$\beta = \frac{\langle \mathbf{e}_2,\, \mathbf{A}_O \times \mathbf{u} \rangle}{\langle \mathbf{u},\, \mathbf{N}_T \rangle}, \quad \gamma = -\frac{\langle \mathbf{e}_1,\, \mathbf{A}_O \times \mathbf{u} \rangle}{\langle \mathbf{u},\, \mathbf{N}_T \rangle}, \quad \alpha = 1 - \beta - \gamma, \quad t = \frac{\langle \mathbf{A}_O,\, \mathbf{N}_T \rangle}{\langle \mathbf{u},\, \mathbf{N}_T \rangle}.$$

The intersection is valid only if $t > 0$ and $\alpha, \beta, \gamma \in [0, 1]$. For each ray, we run this test on every triangle and keep the closest valid hit.

Since most meshes contain thousands of triangles, it is important to think of how we can avoid long runtimes. To speed things up, we added an axis-aligned bounding box (AABB) test as a pre-filter: we compute the overall bounding box of the mesh once, and for each ray we first check if it enters the box using the standard slab method. If it does not, we skip the whole triangle loop.



(a) No bounding box. $512 \times 512$, 16 spp, 5 bounce. Render time: $\approx$ **4min**.

(b) With AABB. $512 \times 512$, 16 spp, 5 bounces. Render time: **48.75 s**.

Figure 3: Lab 3 renders. Both images look virtually the same due to the use of the same parameters, but highlight the speedup when using AABB.

# 4 Lab 4: BVH acceleration

In Lab 4, we replaced the single mesh-level AABB with a bounding volume hierarchy (BVH). The idea is to recursively split the triangles into smaller groups, each with its own bounding box, so that most rays only need to test a small subset triangles instead of all of them. In addition, whereas the AABB of the cat was computed for every ray, creative a BVH class outside of the main computation loops meant that we weren't repeating useless operations!

We build the BVH top-down. For a node covering a range of triangles, we compute its bounding box and stop if it has 5 or fewer triangles (it becomes a leaf). Otherwise we pick the axis along which the bounding box is longest, take the midpoint of that axis as the split, and reorder the triangle indices in place so that those whose centroid lies below the midpoint go to the left child and the others to the right. Then we recurse on each half.

We use traversal in a similar way to what was done in Lab 3 with the AABB, except here, we do it with BVH nodes. We prune and skip nodes/subtrees that aren't reached by the ray, and run Moller–Trumbore on the leaves of the tree. Combined, this leads to the biggest speedup seen across all labs.

Indeed, despite using much heavier parameters than Lab 3 – $1024 \times 1024$ instead of $512 \times 512$, 32 spp instead of 16, and 10 bounces instead of 5, Lab 4 yields a render in **15.71 s** against **48.75 s** for the AABB-sped-up Lab 3 render at one quarter the pixel count, half the sampling, and half the bounce depth.

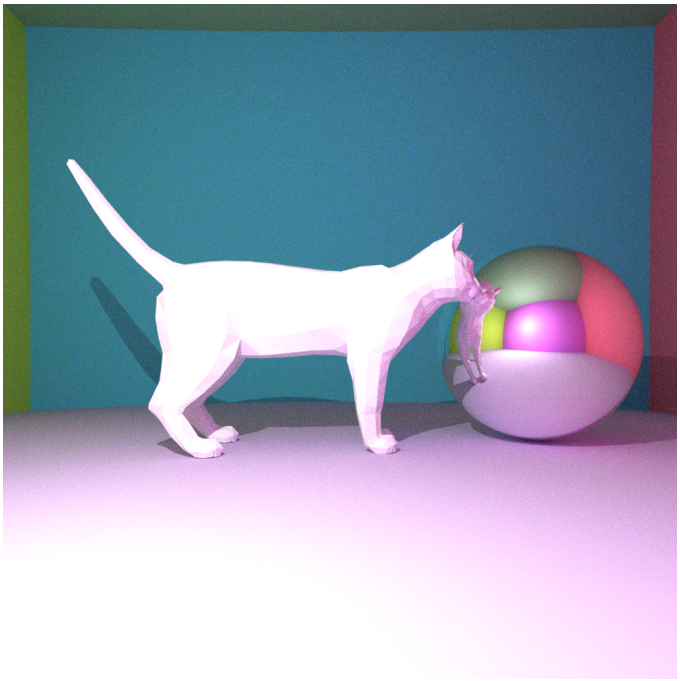


Figure 4: Lab 4 render. $1024 \times 1024$, 32 spp, 10 bounces, Render time: **15.71 s**.

## Conclusion

Over the course of this project, we went from a basic pathtracer/raycaster template to a fully fledged raycaster with support for complex meshes, indirect lighting, mirrored surfaces, antialiasing and optimisation techniques to reduce runtime like BVH. Each lab added an important improvement, pushing our final result closer to realism.

Particularly interesting was the use of BVH, which was a great example of how using data structures like trees can have concrete uses for optimisation of 3D systems.